DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

How Numbers Are Stored

CMSC 131 Bootcamp Block 3

Prepared by: Rene Andre Bedonia Jocsing

Published: September 02, 2026 **Deadline:** September 02, 2026

Prerequisites

One archive holds everything this block needs, including a project that already builds. You can start today without Git, without a GitHub account, and without last session's folder.

You need this archive. Unzip it somewhere permanent and work inside the folder it makes.

Session Objectives

- Read a 32-bit pattern as either a signed or an unsigned number
- Explain two's complement and why it makes subtraction free
- Choose correctly between `div` and `idiv`, and between `mul` and `imul`
- Handle a multiplication whose result outgrows one register
- Recognise overflow when it happens instead of trusting the output

Scoring

This block is worth 10 points for work completed during its scheduled laboratory session. Your instructor checks your progress before the session ends and prorates the 10 points according to how much of the block you completed. Complete all seven guided blocks, `b1` through `b7`, without an absence and you earn a 30-point completion bonus. Attendance is checked during every bootcamp session, so it doesn't carry a separate score.

Before You Start

You need Block 2's working project. The archive above is one, with today's files and Block 2's finished converter already in it, so a session you missed doesn't stop you starting this one.

If you prefer to keep working in your own folder, copy these across instead.

File	What it's for
`b3_starter.asm`	Today's exercise, with the prompts written and the five interesting parts left to you
`print_uint.inc`	A routine you're handed, explained in Part 5
`signs.input`	One number, fed to your program by `make check`
`signs.expected`	What a correct program prints for that number
`b2_solution.asm`	Block 2's temperature converter, finished, if you missed it

Ninety minutes, roughly twenty on two's complement, twenty-five on the two divides, fifteen on multiplication that doesn't fit, and thirty on the exercise.

Part 1: A Register Holds a Pattern, Not a Number

Here is the uncomfortable fact underneath this whole block. A register holds 32 bits. It doesn't hold a number. The bits become a number only when some instruction decides how to read them.

Take this pattern:

```
11111111 11111111 11111111 11111111
```

Read as an unsigned number, that's 4,294,967,295. Read as a signed number, it's -1. The bits are identical. Nothing in the register records which reading you meant. The processor will happily give you either.

So who decides? You. And the instructions you write.

Part 2: Two's Complement

The obvious way to store a negative number is to reserve one bit for the sign. That approach, called signed magnitude, produces two zeroes (`+0` and `-0`) and needs special-cased addition depending on the signs.

Real machines use **two's complement** instead. To negate a number, flip every bit, then add one.

Work through -5 in eight bits:

```
  5          00000101
  flip       11111010
  add 1      11111011      <- this is -5
```

Now add 5 to it, using ordinary binary addition with no special cases:

```
  11111011   (-5)
+ 00000101   ( 5)
-----------
 100000000
```

The ninth bit falls off the end, leaving `00000000`. Zero is the right answer.

Why did hardware designers choose this representation? That falling-off-the-end behaviour is the entire point. The adder circuit that computes `5 + 3` also computes `5 + (-3)` with no modification. Subtraction becomes negate-then-add, so the chip needs one adder rather than an adder plus a subtractor plus sign-comparison logic.

There's a cost, but it's small. The range is lopsided, running from -2,147,483,648 to 2,147,483,647 for 32 bits. Zero occupies one of the non-negative slots. And there's exactly one zero, which is the fix for signed magnitude's other problem.

In NASM you never write out the flipped bits. `neg eax` does it, and `mov eax, -5` assembles to the right pattern. What matters is knowing that the top bit being set means "negative" to any instruction that reads it as signed, and means "a very large value" to any instruction that doesn't.

Part 3: `div` Against `idiv`

Block 2 used `div` and told you to clear `edx` first. Now the reason is clearer. Picking the wrong one of the two divide instructions is a bug waiting to happen.

`div` **is unsigned.** It treats `edx:eax` as a 64-bit non-negative value.

`idiv` **is signed.** It treats `edx:eax` as a 64-bit two's complement value.

That difference changes how you prepare `edx`:

```
; Unsigned: clear edx.
        mov     edx, 0
        mov     eax, 100
        mov     ebx, 7
        div     ebx             ; eax = 14, edx = 2

; Signed: sign-extend eax into edx with cdq.
        mov     eax, -100
        cdq                     ; edx becomes 0xFFFFFFFF because eax is negative
        mov     ebx, 7
        idiv    ebx             ; eax = -14, edx = -2
```

`cdq` (convert doubleword to quadword) copies the sign bit of `eax` across all of `edx`. If `eax` is positive it fills `edx` with zeroes, and if negative, with ones. That's what makes `edx:eax` a correct 64-bit version of the 32-bit number in `eax`.

Cross the two and the mistake is silent. Use `mov edx, 0` before `idiv` on a negative number and you've told the processor the dividend is a large positive value. It won't complain. It'll just return an answer that's wrong by about four billion. Use `cdq` before `div` and it fails the other way. If `eax` was negative, `cdq` fills `edx` with ones and `div` reads the pair as an enormous positive number, which usually overflows and crashes.

The rule is short. `div` pairs with `mov edx, 0`. `idiv` pairs with `cdq`. Never cross them.

Part 4: Multiplication That Doesn't Fit

`mul` writes its result across `edx:eax` for a reason. Two 32-bit numbers can multiply to something that needs 64 bits.

```
        mov     eax, 100000
        mov     ebx, 100000
        mul     ebx
```

The true answer is 10,000,000,000. The largest value a 32-bit register holds is 4,294,967,295. So the answer doesn't fit. The processor splits it. The low 32 bits land in `eax`, the high 32 bits in `edx`.

If you print `eax` alone you get 1,410,065,408. It's confidently wrong.

If `edx` is zero after a `mul`, the answer fits in `eax` and you can use it normally. If `edx` is non-zero, the result overflowed 32 bits and `eax` alone is meaningless.

```
        mul     ebx
        cmp     edx, 0
        jne     overflow_happened
```

Block 5 covers `cmp` and `jne` properly. For now, note the habit. A multiply that could get large deserves a look at `edx` before you trust `eax`.

`imul` is the signed counterpart. It has a convenient two-operand form that writes only to the destination:

```
        imul    eax, ebx          ; eax = eax * ebx, edx untouched
```

That form is safe when you know the result fits. It's also easier to read than plain `mul`.

Part 5: Printing the Unsigned Reading

There's a gap between what this block claims and what you can currently do. Carter's `print_int` hands `printf` the format `%i`, which reads its argument as signed. So `print_int` can show you -100. Nothing in the library can show you the 4,294,967,196 that the same bits spell when read the other way, which makes the central point of this block hard to demonstrate.

So this block hands you a routine. `print_uint.inc` came with the archive. You use it exactly like one of Carter's:

```
%include "asm_io.inc"
%include "print_uint.inc"

        mov     eax, esi
        call    print_uint
```

Put the `%include` after the one for `asm_io.inc`. The routine uses `print_char`, which is declared there. Like Carter's routines, it gives you back every register.

Treat it as a box you were handed. Open it if you're curious, but the loop inside is Block 5 material and you're not expected to follow it yet. The short version is that dividing by ten leaves the last digit behind as a remainder, so the routine peels digits off the bottom one at a time and then prints them back in the order you read them.

Part 6: Exercise

Start from `b3_starter.asm`, which has the prompts written and assembles as it stands, so your first build works before you have written anything.

```
cp b3_starter.asm signs.asm
```

Write a program that reads one integer and reports five things about it.

1. The value read back as a **signed** number, with `print_int`.
2. The same bits read as an **unsigned** number, with `print_uint`.
3. The value divided by 7, quotient and remainder, computed with `idiv`.
4. The value multiplied by 100,000 with `mul`, the unsigned multiply, reported as the product or as `overflowed`.
5. The same multiplication with `imul`, the signed one, reported the same way.

Steps 4 and 5 ask the same arithmetic question twice, of two instructions that answer it differently. The two lines are the payoff of the whole block.

They also need different tests. `mul` spreads its answer across `edx:eax`, so a non-zero `edx` is what tells you the product outgrew 32 bits. The two-operand `imul` keeps its answer in one register and raises the **overflow flag** instead, so `jo` is what asks it. Using `edx` to judge an `imul` doesn't work. `imul eax, ebx` never writes to `edx` at all.

Test it with -100, with 100, and with 100000.

Expected format of output:

```
Enter an integer: -100
As signed:                -100
As unsigned:              4294967196
Divided by 7:             -14 remainder -2
Times 100000 (unsigned): overflowed
Times 100000 (signed):   -10000000

Enter an integer: 100000
As signed:                100000
As unsigned:              100000
Divided by 7:             14285 remainder 5
Times 100000 (unsigned): overflowed
Times 100000 (signed):   overflowed
```

Look at the two multiply lines for -100. Because -10,000,000 is a perfectly ordinary 32-bit signed number, the signed multiply fits comfortably. The unsigned one overflows, since `mul` never saw a negative hundred. It saw 4,294,967,196. Multiplying that by a hundred thousand needs a much bigger register than you have.

The signed and unsigned lines differ for -100 but agree for 100000. What's true of every number where they agree?

Checking it

```
make PROG=signs check
```

`signs.input` holds `-100`, so that's the case being checked. The other two you run by hand.

Testing Checklist

Core Functionality

- A positive input prints the same value on the signed and unsigned lines
- A negative input prints a very large value on the unsigned line
- `idiv` on a negative input gives a negative quotient and a negative remainder
- An input of -100 overflows the unsigned multiply and not the signed one
- An input of 100000 overflows both
- An input of 100 overflows neither, and both product lines read 10000000
- The program never crashes with a divide error
- `make PROG=signs check` prints `OK: signs matches signs.expected`

Common Pitfalls

- Pairing `cdq` with `div`, or `mov edx, 0` with `idiv`
- Forgetting that `mul` overwrites `edx` even when nothing overflowed
- Reading `eax` after an overflowing multiply and believing the number
- Testing `edx` after an `imul`, which never wrote there, so the test reads whatever the last `mul` left behind
- Putting an instruction between `imul` and `jo`, which can clear the flag before you ask about it
- Assuming `print_int` shows the unsigned reading, when it prints signed
- Overwriting the original value with the first multiply, so the second one multiplies the wrong number
- Expecting a remainder to be positive when the dividend was negative

Architecture Review

What We Built

- A program that shows one bit pattern under two different readings
- Correct signed and unsigned division, each with its proper `edx` setup
- Two overflow checks, each asking the question the way its instruction answers it

Key Takeaways

1. **Bits carry no sign.** The instruction supplies the interpretation.
2. **Two's complement exists so one adder can do subtraction.** The asymmetric range is the price.
3. `div` **with** `mov edx, 0`**,** `idiv` **with** `cdq`**.** Crossing them fails silently.
4. `mul` **produces 64 bits and** `imul` **sets a flag.** Ask `edx` about one and `jo` about the other.
5. **The same multiplication overflows or doesn't depending on signedness.** -100 proves it in one run.
6. **Nothing warns you.** Wrong-signedness produces plausible numbers.

Next Session Preview

- Reading a crash instead of guessing at it
- `dump_regs`, and seeing every register at once
- Running your program under `gdb`, one instruction at a time
- Breakpoints, and stopping right before the thing that goes wrong